JAVA COLLECTION FRAMEWORK

Practical Programs and Execution Report

Combined report containing 11 Java examples covering Collection, List, Set, Queue, Deque, Map, Iterator and related interfaces.

Included in each program
Aim / Objective
System Requirements
Algorithm
Interface / Class Relationship (UML-style)
Source Code
Compilation and Execution Commands
Output
Result

Note: output panels are formatted terminal-style representations. They are not screenshots captured from the user's computer.

Contents

1. Program 1: CollectionExample
2. Program 2: DequeExample
3. Program 3: IteratorExample
4. Program 4: ListExample
5. Program 5: ListIteratorExample
6. Program 6: MapExample
7. Program 7: NavigableSetExample
8. Program 8: QueueExample
9. Program 9: SortedMapExample
10. Program 10: SortedSetExample
11. Program 11: SetExample

Program 1: CollectionExample

Aim / Objective

To demonstrate basic operations on an ArrayList, including adding, accessing, removing, and counting elements.

System Requirements

JDK 8 or later; a text editor or Java IDE; command-line terminal.

Algorithm

1. Create an ArrayList of student names.
2. Add four student names.
3. Display the list and access the first student.
4. Remove Ravi from the list.
5. Display the updated list and its size.

Interface / Class Relationship (UML-style)

```
+---------------------------+
| CollectionExample |
+---------------------------+
| main(String[] args)       |
+---------------------------+
        | uses
        v
ArrayList; Collection operations add(), get(), remove(), size().
```

Source Code

```
import java.util.ArrayList;

public class CollectionExample {
    public static void main(String[] args) {
        ArrayList<String> students = new ArrayList<>();
        // Adding elements
        students.add("Anu");
        students.add("Ravi");
        students.add("Priya");
        students.add("Kiran");
        // Display collection
        System.out.println("Students List: " + students);
        // Accessing an element
        System.out.println("First Student: " + students.get(0));
        // Removing an element
        students.remove("Ravi");
        // Display after removal
        System.out.println("After Removal: " + students);
        // Size of collection
        System.out.println("Total Students: " + students.size());
    }
}
```

Compilation and Execution Commands

```
javac CollectionExample.java
java CollectionExample
```

Output

```
Students List: [Anu, Ravi, Priya, Kiran]
First Student: Anu
After Removal: [Anu, Priya, Kiran]
Total Students: 3
```

Result

Compiled and executed successfully in the report build. ArrayList; Collection operations add(), get(), remove(), size().

Program 2: DequeExample

Aim / Objective

To demonstrate insertion and removal at both ends of a double-ended queue.

System Requirements

JDK 8 or later; a text editor or Java IDE; command-line terminal.

Algorithm

1. Create a Deque using ArrayDeque.
2. Insert orders at the front and rear.
3. Display the queue.
4. Remove one order from the front and one from the rear.
5. Display the remaining orders.

Interface / Class Relationship (UML-style)

```
+---------------------------+
| DequeExample |
+---------------------------+
| main(String[] args)       |
+---------------------------+
        | uses
        v
Deque interface; ArrayDeque; addFirst(), addLast(), removeFirst(), removeLast().
```

Source Code

```
import java.util.ArrayDeque;
import java.util.Deque;

public class DequeExample {
    public static void main(String[] args) {
        Deque<String> orders = new ArrayDeque<>();
        // Adding orders
        orders.addFirst("Coffee");
        orders.addLast("Burger");
        orders.addFirst("Tea");
        orders.addLast("Pizza");
        System.out.println("Café Orders: " + orders);
        // Remove order from front
        System.out.println("Completed Order: " + orders.removeFirst());
        // Remove order from rear
        System.out.println("Cancelled Order: " + orders.removeLast());
        System.out.println("Remaining Orders: " + orders);
    }
}
```

Compilation and Execution Commands

```
javac DequeExample.java
java DequeExample
```

Output

```
Café Orders: [Tea, Coffee, Burger, Pizza]
Completed Order: Tea
Cancelled Order: Pizza
Remaining Orders: [Coffee, Burger]
```

Result

Compiled and executed successfully in the report build. Deque interface; ArrayDeque; addFirst(), addLast(), removeFirst(), removeLast().

Program 3: IteratorExample

Aim / Objective

To traverse an ArrayList using an Iterator.

System Requirements

JDK 8 or later; a text editor or Java IDE; command-line terminal.

Algorithm

1. Create an ArrayList of movie titles.

2. Add four movies.

3. Create an Iterator from the list.

4. Use hasNext() and next() to print each movie.

Interface / Class Relationship (UML-style)

```
+---------------------------+
| IteratorExample |
+---------------------------+
| main(String[] args)       |
+---------------------------+
        | uses
        v
Iterator; iterator(), hasNext(), next().
```

Source Code

```
import java.util.ArrayList;
import java.util.Iterator;

public class IteratorExample {
    public static void main(String[] args) {
        ArrayList<String> movies = new ArrayList<>();
        movies.add("Pushpa");
        movies.add("Jersey");
        movies.add("Sita Ramam");
        movies.add("Hi Nanna");

        Iterator<String> it = movies.iterator();
        System.out.println("My Watchlist:");
        while (it.hasNext()) {
            System.out.println(it.next());
        }
    }
}
```

Compilation and Execution Commands

```
javac IteratorExample.java
java IteratorExample
```

Output

```
My Watchlist:
Pushpa
Jersey
Sita Ramam
Hi Nanna
```

Result

Compiled and executed successfully in the report build. Iterator; iterator(), hasNext(), next().

Program 4: ListExample

Aim / Objective

To demonstrate List operations using an ArrayList.

System Requirements

JDK 8 or later; a text editor or Java IDE; command-line terminal.

Algorithm

1. Create a List of places backed by ArrayList.
2. Add four places.
3. Display the list and access its first element.
4. Remove Goa.
5. Display the updated list and size.

Interface / Class Relationship (UML-style)

```
+---------------------------+
| ListExample |
+---------------------------+
| main(String[] args)       |
+---------------------------+
        | uses
        v
List interface; ArrayList; add(), get(), remove(), size().
```

Source Code

```
import java.util.ArrayList;
import java.util.List;

public class ListExample {
    public static void main(String[] args) {
        List<String> places = new ArrayList<>();
        places.add("Kerala");
        places.add("Goa");
        places.add("Ooty");
        places.add("Jaipur");

        System.out.println("My Travel List: " + places);
        // Accessing an element
        System.out.println("First Place: " + places.get(0));
        // Removing an element
        places.remove("Goa");
        System.out.println("After Removal: " + places);
        // Size of List
        System.out.println("Total Places: " + places.size());
    }
}
```

Compilation and Execution Commands

```
javac ListExample.java
java ListExample
```

Output

```
My Travel List: [Kerala, Goa, Ooty, Jaipur]
First Place: Kerala
After Removal: [Kerala, Ooty, Jaipur]
Total Places: 3
```

Result

Compiled and executed successfully in the report build. List interface; ArrayList; add(), get(), remove(), size().

Program 5: ListIteratorExample

Aim / Objective

To traverse a list in both forward and reverse directions using ListIterator.

System Requirements

JDK 8 or later; a text editor or Java IDE; command-line terminal.

Algorithm

1. Create a List of song titles.

2. Add four songs.

3. Use ListIterator to print songs forward.

4. Use the same iterator to print songs in reverse.

Interface / Class Relationship (UML-style)

```
+---------------------------+
| ListIteratorExample |
+---------------------------+
| main(String[] args)       |
+---------------------------+
        | uses
        v
ListIterator; listIterator(), hasNext(), next(), hasPrevious(), previous().
```

Source Code

```
import java.util.ArrayList;
import java.util.List;
import java.util.ListIterator;

public class ListIteratorExample {
    public static void main(String[] args) {
        List<String> songs = new ArrayList<>();
        songs.add("Kesariya");
        songs.add("Samajavaragamana");
        songs.add("Inkem Inkem");
        songs.add("Butta Bomma");

        ListIterator<String> it = songs.listIterator();
        System.out.println("Songs in Forward Direction:");
        while (it.hasNext()) {
            System.out.println(it.next());
        }
        System.out.println("\nSongs in Reverse Direction:");
        while (it.hasPrevious()) {
            System.out.println(it.previous());
        }
    }
}
```

Compilation and Execution Commands

```
javac ListIteratorExample.java
java ListIteratorExample
```

Output

```
Songs in Forward Direction:
Kesariya
Samajavaragamana
Inkem Inkem
Butta Bomma

Songs in Reverse Direction:
Butta Bomma
Inkem Inkem
Samajavaragamana
Kesariya
```

Result

Compiled and executed successfully in the report build. ListIterator; listIterator(), hasNext(), next(), hasPrevious(), previous().

Program 6: MapExample

Aim / Objective

To demonstrate key-value mapping with HashMap, including lookup and removal.

System Requirements

JDK 8 or later; a text editor or Java IDE; command-line terminal.

Algorithm

1. Create a Map of route names and bus numbers.

2. Insert four entries.

3. Print the map and retrieve one value using its key.

4. Remove one entry and display the updated map.

Interface / Class Relationship (UML-style)

```
+---------------------------+
| MapExample |
+---------------------------+
| main(String[] args)       |
+---------------------------+
        | uses
        v
Map interface; HashMap; put(), get(), remove(). HashMap iteration order is unspecified.
```

Source Code

```
import java.util.HashMap;
import java.util.Map;

public class MapExample {
    public static void main(String[] args) {
        Map<String, Integer> busRoutes = new HashMap<>();
        busRoutes.put("Vijayawada-Guntur", 101);
        busRoutes.put("Vijayawada-Hyderabad", 205);
        busRoutes.put("Vijayawada-Vizag", 310);
        busRoutes.put("Guntur-Tirupati", 415);

        System.out.println("Bus Routes: " + busRoutes);
        // Accessing value using key
        System.out.println("Bus for Vijayawada-Vizag: "
                + busRoutes.get("Vijayawada-Vizag"));
        // Removing an entry
        busRoutes.remove("Guntur-Tirupati");
        System.out.println("After Removal: " + busRoutes);
    }
}
```

Compilation and Execution Commands

```
javac MapExample.java
java MapExample
```

Output

```
Bus Routes: {Vijayawada-Hyderabad=205, Guntur-Tirupati=415, Vijayawada-Vizag=310, Vijayawada-Guntur=101}
Bus for Vijayawada-Vizag: 310
After Removal: {Vijayawada-Hyderabad=205, Vijayawada-Vizag=310, Vijayawada-Guntur=101}
```

Result

Compiled and executed successfully in the report build. Map interface; HashMap; put(), get(), remove(). HashMap iteration order is unspecified.

Program 7: NavigableSetExample

Aim / Objective

To demonstrate navigation operations on a sorted set.

System Requirements

JDK 8 or later; a text editor or Java IDE; command-line terminal.

Algorithm

1. Create a NavigableSet backed by TreeSet.
2. Add five temperatures.
3. Display the sorted set.
4. Retrieve the lowest and highest values.
5. Find the closest values strictly below and above 32.

Interface / Class Relationship (UML-style)

```
+---------------------------+
| NavigableSetExample |
+---------------------------+
| main(String[] args)       |
+---------------------------+
        | uses
        v
NavigableSet; TreeSet; first(), last(), lower(), higher().
```

Source Code

```
import java.util.NavigableSet;
import java.util.TreeSet;

public class NavigableSetExample {
    public static void main(String[] args) {
        NavigableSet<Integer> temperatures = new TreeSet<>();
        temperatures.add(32);
        temperatures.add(28);
        temperatures.add(35);
        temperatures.add(30);
        temperatures.add(25);

        System.out.println("Temperatures: " + temperatures);
        System.out.println("Lowest Temperature: "
                + temperatures.first());
        System.out.println("Highest Temperature: "
                + temperatures.last());
        System.out.println("Temperature below 32: "
                + temperatures.lower(32));
        System.out.println("Temperature above 32: "
                + temperatures.higher(32));
    }
}
```

Compilation and Execution Commands

```
javac NavigableSetExample.java
java NavigableSetExample
```

Output

```
Temperatures: [25, 28, 30, 32, 35]
Lowest Temperature: 25
Highest Temperature: 35
Temperature below 32: 30
Temperature above 32: 35
```

Result

Compiled and executed successfully in the report build. NavigableSet; TreeSet; first(), last(), lower(), higher().

Program 8: QueueExample

Aim / Objective

To demonstrate FIFO queue operations using LinkedList.

System Requirements

JDK 8 or later; a text editor or Java IDE; command-line terminal.

Algorithm

1. Create a Queue of patient names backed by LinkedList.
2. Add four patients.
3. Use peek() to inspect the next patient.
4. Use poll() to remove the first patient.
5. Display the remaining queue.

Interface / Class Relationship (UML-style)

```
+---------------------------+
| QueueExample |
+---------------------------+
| main(String[] args)       |
+---------------------------+
        | uses
        v
Queue; LinkedList; add(), peek(), poll(); FIFO (First In, First Out).
```

Source Code

```
import java.util.LinkedList;
import java.util.Queue;

public class QueueExample {
    public static void main(String[] args) {
        Queue<String> patients = new LinkedList<>();
        patients.add("Ravi");
        patients.add("Anu");
        patients.add("Kiran");
        patients.add("Priya");

        System.out.println("Patient Queue: " + patients);
        // First patient
        System.out.println("Next Patient: " + patients.peek());
        // Removing first patient
        System.out.println("Consulted Patient: " + patients.poll());
        System.out.println("Remaining Patients: " + patients);
    }
}
```

Compilation and Execution Commands

```
javac QueueExample.java
java QueueExample
```

Output

```
Patient Queue: [Ravi, Anu, Kiran, Priya]
Next Patient: Ravi
Consulted Patient: Ravi
Remaining Patients: [Anu, Kiran, Priya]
```

Result

Compiled and executed successfully in the report build. Queue; LinkedList; add(), peek(), poll(); FIFO (First In, First Out).

Program 9: SortedMapExample

Aim / Objective

To demonstrate sorted key-value mappings and retrieval of boundary keys.

System Requirements

JDK 8 or later; a text editor or Java IDE; command-line terminal.

Algorithm

1. Create a SortedMap backed by TreeMap.

2. Insert four flight codes and departure times.

3. Display the map and retrieve one departure time.

4. Print the first and last keys.

Interface / Class Relationship (UML-style)

```
+---------------------------+
| SortedMapExample |
+---------------------------+
| main(String[] args)       |
+---------------------------+
        | uses
        v
SortedMap; TreeMap; put(), get(), firstKey(), lastKey(). Keys are sorted naturally.
```

Source Code

```
import java.util.SortedMap;
import java.util.TreeMap;

public class SortedMapExample {
    public static void main(String[] args) {
        SortedMap<String, String> flights = new TreeMap<>();
        flights.put("HYD101", "06:30 AM");
        flights.put("VJA205", "08:00 AM");
        flights.put("DEL310", "10:15 AM");
        flights.put("BLR415", "07:45 AM");

        System.out.println("Flight Schedule: " + flights);
        // Getting value using key
        System.out.println("Departure of VJA205: "
                + flights.get("VJA205"));
        // First key
        System.out.println("First Flight: "
                + flights.firstKey());
        // Last key
        System.out.println("Last Flight: "
                + flights.lastKey());
    }
}
```

Compilation and Execution Commands

```
javac SortedMapExample.java
java SortedMapExample
```

Output

```
Flight Schedule: {BLR415=07:45 AM, DEL310=10:15 AM, HYD101=06:30 AM, VJA205=08:00 AM}
Departure of VJA205: 08:00 AM
First Flight: BLR415
Last Flight: VJA205
```

Result

Compiled and executed successfully in the report build. SortedMap; TreeMap; put(), get(), firstKey(), lastKey(). Keys are sorted naturally.

Program 10: SortedSetExample

Aim / Objective

To demonstrate a sorted set and its boundary and size operations.

System Requirements

JDK 8 or later; a text editor or Java IDE; command-line terminal.

Algorithm

1. Create a SortedSet backed by TreeSet.

2. Add five parking slot numbers.

3. Display the set in sorted order.

4. Print the first slot, last slot, and number of elements.

Interface / Class Relationship (UML-style)

```
+---------------------------+
| SortedSetExample |
+---------------------------+
| main(String[] args)       |
+---------------------------+
        | uses
        v
SortedSet; TreeSet; add(), first(), last(), size().
```

Source Code

```
import java.util.SortedSet;
import java.util.TreeSet;

public class SortedSetExample {
    public static void main(String[] args) {
        SortedSet<Integer> parkingSlots = new TreeSet<>();
        parkingSlots.add(25);
        parkingSlots.add(10);
        parkingSlots.add(40);
        parkingSlots.add(15);
        parkingSlots.add(30);

        System.out.println("Parking Slots: " + parkingSlots);
        System.out.println("First Slot: "
                + parkingSlots.first());
        System.out.println("Last Slot: "
                + parkingSlots.last());
        System.out.println("Total Slots: "
                + parkingSlots.size());
    }
}
```

Compilation and Execution Commands

```
javac SortedSetExample.java
java SortedSetExample
```

Output

```
Parking Slots: [10, 15, 25, 30, 40]
First Slot: 10
Last Slot: 40
Total Slots: 5
```

Result

Compiled and executed successfully in the report build. SortedSet; TreeSet; add(), first(), last(), size().

Program 11: SetExample

Aim / Objective

To demonstrate a Set that stores unique values and supports membership testing and removal.

System Requirements

JDK 8 or later; a text editor or Java IDE; command-line terminal.

Algorithm

1. Create a Set backed by HashSet.

2. Add book codes, including a duplicate.

3. Display the set and check whether BK205 exists.

4. Remove BK310 and display the updated set.

Interface / Class Relationship (UML-style)

```
+---------------------------+
| SetExample |
+---------------------------+
| main(String[] args)       |
+---------------------------+
        | uses
        v
Set; HashSet; add(), contains(), remove(). HashSet does not guarantee display order.
```

Source Code

```
import java.util.HashSet;
import java.util.Set;

public class SetExample {
    public static void main(String[] args) {
        Set<String> bookCodes = new HashSet<>();
        bookCodes.add("BK101");
        bookCodes.add("BK205");
        bookCodes.add("BK310");
        bookCodes.add("BK101");

        System.out.println("Book Codes: " + bookCodes);
        // Checking an element
        System.out.println("Is BK205 available? "
                + bookCodes.contains("BK205"));
        // Removing an element
        bookCodes.remove("BK310");
        System.out.println("After Removal: " + bookCodes);
    }
}
```

Compilation and Execution Commands

```
javac SetExample.java
java SetExample
```

Output

```
Book Codes: [BK205, BK101, BK310]
Is BK205 available? true
After Removal: [BK205, BK101]
```

Result

Compiled and executed successfully in the report build. Set; HashSet; add(), contains(), remove(). HashSet does not guarantee display order.